

TP 13 — Le contrôle qualité

Semaine 13 ■ Fichiers dans NSI/TP13/ ■ 2 h

Objectifs

- Pratiquer le cycle : docstring → tests → corps → vert
- Démasquer des fonctions buguées uniquement grâce à des tests
- Équiper d'anciennes fonctions avec assertions et jeux de tests

Partie A — Le laboratoire de tests — suspects.py

(45 min)

Voici quatre fonctions « prêtes à l'emploi ». Certaines sont correctes, d'autres non. **Interdiction de corriger avant d'avoir un test rouge!** Pour chacune : écrire un jeu de tests complet (cas normaux + limites), l'exécuter, et conclure « correcte » ou « buguée » (avec le test qui accuse).

```
def valeur_absolue(x):  
    """Renvoie la valeur absolue du nombre x."""  
    if x < 0:  
        return -x  
    if x > 0:  
        return x  
  
def derniere_occurrence(t, x):  
    """Renvoie l'indice de la DERNIERE occurrence de x dans t,  
    ou None si absent."""  
    for i in range(len(t) - 1, -1, -1):  
        if t[i] == x:  
            return i  
    return None  
  
def arrondi_dizaine(n):  
    """Renvoie n arrondi à la dizaine la plus proche  
    (n entier >= 0 ; 5 arrondit vers le haut)."""  
    if n % 10 >= 5:  
        return n + (10 - n % 10)  
    return n - n % 10  
  
def nb_mots(phrase):  
    """Renvoie le nombre de mots (séparés par des espaces)."""  
    compteur = 1  
    for c in phrase:  
        if c == " ":  
            compteur = compteur + 1  
    return compteur
```

Partie B — Rénovation — renovation.py

(40 min)

Reprendre **trois** de vos fonctions des TP 9 à 12 (au choix : moyenne, vendre, serie_chaude, victoire, eclaircir...) et les mettre aux normes :

1. docstring complète (rôle, préconditions, modifie ou non l'argument);
2. **assert** de préconditions en tête de corps;
3. fonction **test_xxx()** avec 5 asserts minimum, dont les limites;
4. tout doit passer au vert.

Partie C — Développement piloté par les tests ★★ — fusion.py

(30 min)

On veut `intercale(t1, t2)` : `t1` et `t2` étant **triés**, renvoyer la liste triée de tous leurs éléments (`intercale([1, 4], [2, 3]) → [1, 2, 3, 4]`).

1. Écrire docstring et jeu de tests **d’abord** (listes vides ! tailles différentes ! doublons !).
2. Implémenter avec deux indices `i1`, `i2` qui avancent dans chaque liste (pas de tri!).
3. Au vert ? Vous venez d’écrire la **fusion**, cœur d’un grand algorithme de tri (teaser Terminale) — et l’outil des tables de la semaine 20.